

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
FACULTAD DE INGENIERÍA • DEPARTAMENTO DE SISTEMAS
BASES DE DATOS I

INFORME TÉCNICO DE AULA RTF4
IMPLEMENTACIÓN Y REFINAMIENTO DEL MODELO
LÓGICO DDL

*Motor Relacional de Gestión Económica, Trazabilidad Contable e Inmutabilidad Transaccional
para Servidores de Rol GTA (Economy RP)*

Proyecto de Aula:	Economy RP (Motor Económico Relacional)
Equipo de Trabajo:	Grupo 4
Docente Asesor:	Juan Camilo Palacio Alcaraz
Integrantes:	• Miguel Ángel Cardona Agudelo • Luisa María López Orrego • Jorge Luis Ordoñez Ávila
Materia / Semestre:	Bases de Datos I • Semestre 2026-2
Fecha de Entrega:	03 de Octubre de 2026

RÚBRICA DE EVALUACIÓN OFICIAL RTF4 (100% PONDERADO)	PESO
1. Evolución y Refinamiento del Modelo Lógico (Antes vs. Después)	25% (1.25)
2. Definición y Estructura del Esquema DDL Tabular	25% (1.25)
3. Informe de Mejoras y Proceso de Normalización (1FN, 2FN, 3FN, BCNF, 4FN, DKNF)	20% (1.00)
4. Pruebas Estructurales, Hallazgos y Manejo de Errores de Motor SGBD	20% (1.00)
5. Estructura, Formato Institucional y Defensa Técnica del Modelo	10% (0.50)

1. Objetivo de la Entrega

El objetivo principal de esta fase es la materialización y refinamiento del modelo lógico relacional en el Sistema Gestor de Bases de Datos (MySQL 8.0 con motor transaccional InnoDB), a partir del diseño conceptual previo. Se busca implementar y certificar la robustez del esquema DDL físico para validar la integridad referencial, suprimir inconsistencias y asegurar el estricto cumplimiento de las reglas del negocio de rol, resolviendo fallas críticas tales como:

- **Inconsistencias en tipos de datos:** Estandarización a DECIMAL(12,2) para evitar pérdidas de precisión por coma flotante.
- **Definición rigurosa de claves primarias y foráneas:** Eliminación de claves compuestas ambiguas y soporte de llaves subrogadas.
- **Restricciones de integridad y dominio (CHECKs):** Blindaje de estados y tipos contables para evitar datos huérfanos o no tipificados.
- **Consistencia transaccional y atomicidad (ACID):** Asegurar que en negociaciones de intercambio P2P y liquidación laboral ningún bien ni saldo quede duplicado o en el limbo.
- **Inmutabilidad en el libro mayor contable:** Estructuración de un historial de transacciones *append-only* con partida doble.
- **Automatización de políticas económicas:** Control en tiempo real de límites de posesión por jugador, periodos de enfriamiento laboral, tasas de drenaje anti-inflacionario y detección reactiva de bancarota.

2. Reglas de Negocio (RN001 - RN012)

A continuación se detallan las reglas de negocio formalizadas para el dominio económico del servidor de rol GTA (Economy RP), expresando su condición lógica y su mecanismo de implementación exclusivo en la base de datos:

CÓDIGO	NOMBRE	DESCRIPCIÓN FUNCIONAL	FÓRMULA / CONDICIÓN	IMPACTO DDL / SGBD
RN001	Registro Único Laboral	Al completar una jornada, el sistema calcula y acredita automáticamente la ganancia a la cuenta del jugador.	$Ganancia = Tarifa_base \times Horas$	Tabla jornadas_laborales, PK subrogada id_jornada y columna histórica inmutable monto_pagado.
RN002	Propiedad Única y Exclusiva	Un bien físico (vehículo con placa, propiedad) solo puede pertenecer a un único personaje a la vez.	$Propietario = 1$ por cada Bien activo	Clave foránea atómica items.id_jugador con integridad referencial directa (1:N estricto).
RN003	Inmutabilidad del Historial	Toda transacción financiera o comercial registrada no puede ser editada ni eliminada bajo ninguna circunstancia.	Política Append-Only (Sin UPDATE / DELETE)	Tablas transacciones y detalles_transacciones inmutables protegidas por disparadores.
RN004	Restricción por Deuda / Embargo	Un bien no puede transferirse si posee deuda activa, embargo o cuotas financieras pendientes.	Restriccion = 'Ninguna' (tiene_deuda = FALSE)	Columna tiene_deuda BOOLEAN NOT NULL DEFAULT FALSE y validación en procedimiento de tradeo.
RN005	Trazabilidad de Creación de Dinero	Todo dinero inyectado (salarios, bonos iniciales) debe originarse formalmente de la cuenta oficial 'SISTEMA'.	Origen = Cuenta_Sistema si Emisor = Sistema	cuentas.tipo_cuenta = 'SISTEMA' (id_jugador NULL) y asiento de partida doble en detalles.
RN006	Comisión por Transferencia	Toda compraventa o intercambio descuenta una comisión porcentual que se acredita a la Tesorería como drenaje anti-inflación.	$Comisión = Monto \times Porcentaje_Comisión$	Atributo servidores.porcentaje_comision y asiento secundario de drenaje fiscal en detalles.
RN007	Límite Máximo de Propiedades	Un jugador no puede registrar más bienes que el tope máximo parametrizado por el administrador.	$COUNT(Items) \leq Limite_Bienes_Servidor$	Atributo servidores.limite_bienes_por_jugador y control de aforo en inserción/tradeo.
RN008	Enfriamiento Laboral (Cooldown)	Un jugador debe cumplir un tiempo mínimo de descanso entre turnos del mismo empleo para evitar explotación.	$Hora_Actual - Hora_Ultima \geq Tiempo_Cooldown$	Atributo servidores.tiempo_enfriamiento_min e inspección temporal en inserción laboral.
RN009	Penalización por Bancarota	Si el saldo disponible no cubre obligaciones o queda en negativo, el jugador pasa a estado 'BANCAROTA', bloqueando operaciones.	$Saldo < 0 \Rightarrow Estado = 'BANCAROTA'$	Restricción CHECK (estado IN ('ACTIVO','BANCAROTA','SUSPENDIDO')) en jugadores.
RN010	Confirmación Doble en Tradeo	Ambas partes deben confirmar explícitamente los términos. Cualquier modificación reinicia los candados de confirmación.	Ejecutar si (conf_j1 = TRUE AND conf_j2 = TRUE)	Banderas booleanas confirmacion_j1 y confirmacion_j2 en negociaciones_tradeos.

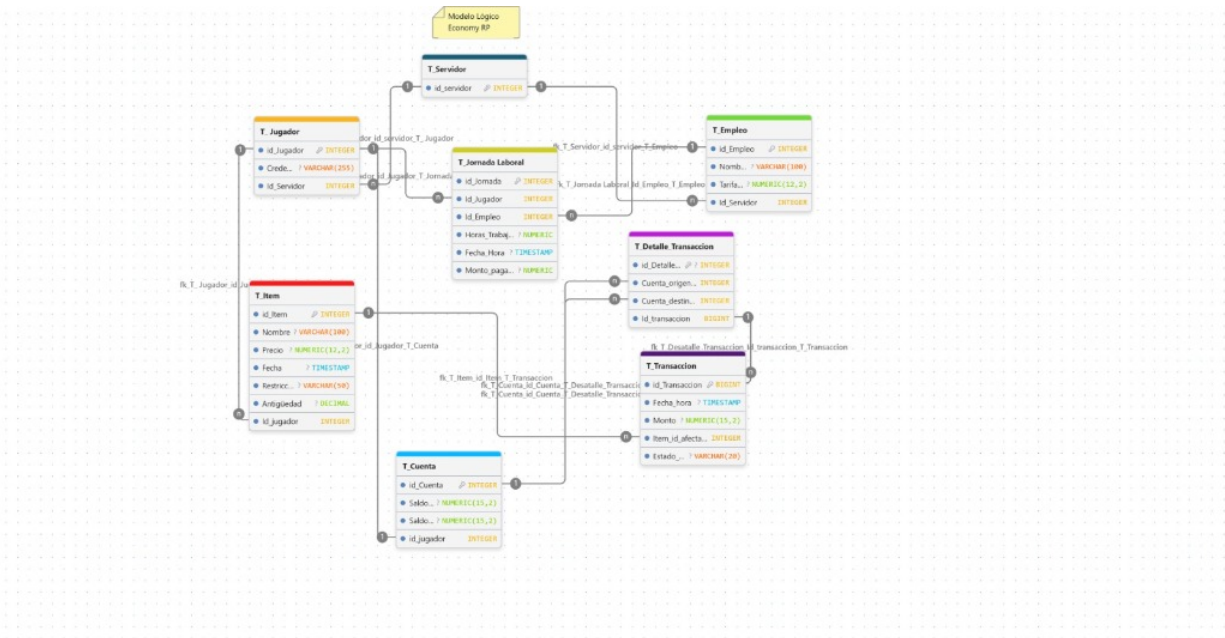
RN011	Límite Diario de Tradeos	Un operador no puede ejecutar más intercambios exitosos al día que el cupo configurado, evitando lavado de activos.	$\text{COUNT}(\text{Tradeos_Hoy}) \leq \text{limite_tradeos_diarios}$	Columna servidores.limite_tradeos_diarios y agregación temporal sobre DATE(fecha_hora).
RN012	Tradeo Único Simultáneo	Un jugador solo puede tener una negociación abierta a la vez, impidiendo comprometer saldos o ítems en paralelo.	$\text{COUNT}(\text{Negociaciones_Activas}) \leq 1$	Restricción de dominio sobre el ciclo de vida CHECK (estado IN (...)) en negociaciones.

3. Evolución del Modelo Lógico

3.1 Modelo Lógico Inicial (Antes)

En la fase conceptual y lógica preliminar del RTF3, la tabla T_Jornada_Laboral contenía el atributo calculado ganancia_calculada lo cual era redundante. Adicionalmente no se especificaba claramente restricciones de unicidad (UNIQUE) en la relación T_Cuenta y T_Jugador, lo que permitía que se pudiera asignar múltiples cuentas personales a un mismo jugador o dejar cabeceras de transacciones sin vinculación directa al origen de la oferta.

Cabe resaltar que tampoco estábamos respetando los parámetros de una nomenclatura purista como siempre lo ha exigido el docente.



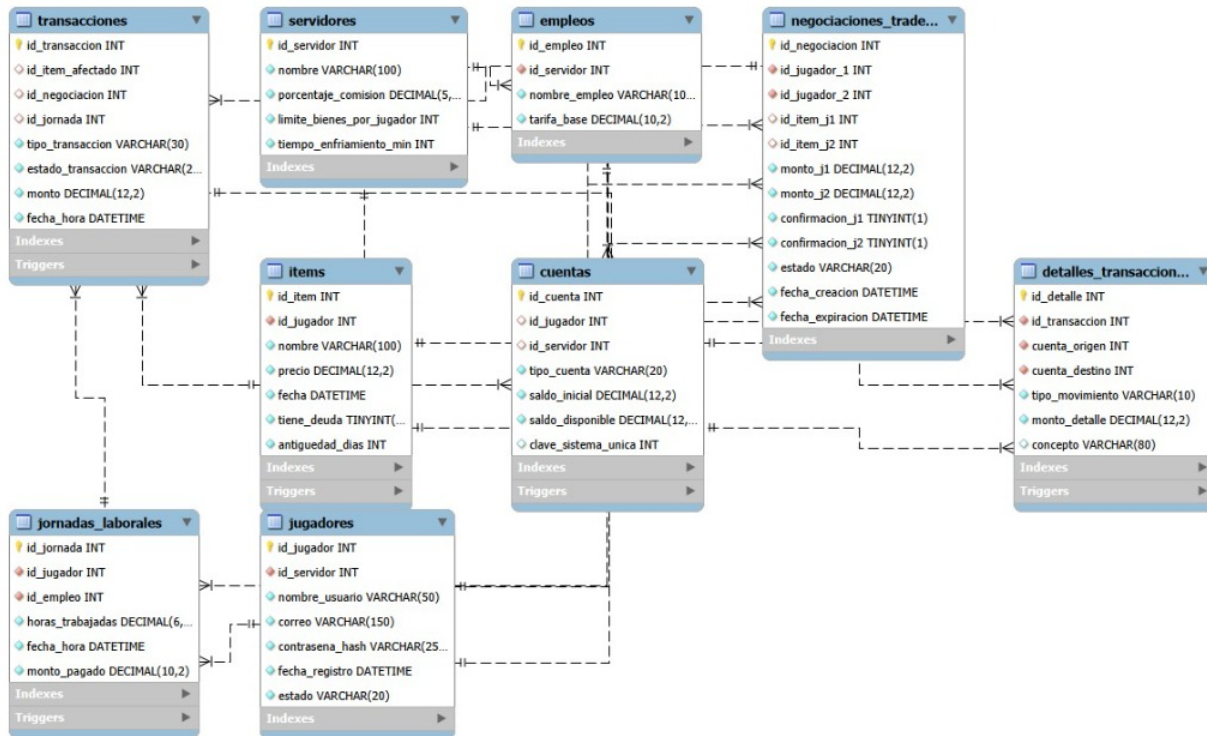
(Primer diseño lógico creado con el motor online DrawDB)



Enlace a aplicación: <https://dbdiagram.io/d/6aa9e23baf7c3b0bd1e995fe>

3.2 Modelo Lógico Corregido (Después)

Aquí se muestra el modelo lógico en el gestor de bases de datos MYSQL tras el proceso de implementación física e inserción DDL.



(Diseño corregido creado en MYSQL)

Este script DDL definitivo consolida la estructura física corrigiendo nomenclaturas, claves y lógica relacional:

- 1. Normalización de Nomenclatura:** Sin prefijos redundantes en tablas: Eliminar el prefijo T_ (las tablas se reconocen por su contexto y definición). Nombres de tablas en plural e identificadores en singular: servidores, jugadores, etc. Además de formato visual consistente (snake_case en minúsculas) para evitar problemas de sensibilidad a mayúsculas/minúsculas entre motores.
- 2. Intercambio Bilateral Simétrico:** negociaciones_tradeos evoluciona para soportar id_item_j1, id_item_j2, monto_j1 y monto_j2, respaldado por la restricción que exige al menos un ítem o monto en la oferta.
- 3. Gestión de Cuenta del Sistema:** Se habilitó la cuenta oficial con tipo_cuenta = 'SISTEMA' (id_jugador NULL), garantizando estrictamente una sola cuenta de Sistema por cada servidor de juego para la inyección lícita salarial.
- 4. Capa Transaccional y Eventos Automáticos:** Se automatizaron los cobros y transferencias vía procedimientos y bloques transaccionales atómicos (ACID), forzando la inmutabilidad contable mediante disparadores y eventos de expiración temporal.

ENTIDAD	DISEÑO INICIAL (RTF3 / ANTES)	DISEÑO CORREGIDO (RTF4 / DESPUÉS)	MEJORA TÉCNICA APORTADA
servidores	Contenedor conceptual sin atributos claros de gobernanza.	Tabla con porcentaje_comision, limite_bienes, cooldown_min y limite_tradeos_diarios.	Centraliza la parametrización de RN006, RN007, RN008 y RN011.
jugadores	Atributos genéricos, credenciales en texto plano, sin restricción de estado.	PK simple, FK a servidores, username/correo únicos, hash criptográfico y CHECK (estado IN (...)).	Integridad referencial, seguridad de acceso y soporte formal de estados judiciales (RN009).
empleos	Mezclaba horas trabajadas con la definición del oficio.	Catálogo desacoplado con id_empleo PK, FK a servidor, nombre y tarifa_base DECIMAL(10,2).	Desacoplamiento total entre la oferta laboral y los turnos individuales ejecutados.
jornadas_laborales	Planteada con PK compuesta (jugador + empleo) y ganancia calculada dinámica.	PK subrogada id_jornada, FKs independientes, horas trabajadas y monto_pagado inmutable.	Cumplimiento estricto de 2FN y 3FN; historial laboral auditable sin desincronización de tarifas.
cuentas	Atada 1:1 rígidamente a Jugador con UNIQUE. Sin cuenta oficial del sistema.	PK simple, FK a jugador (nullable), FK a servidor, tipo_cuenta CHECK ('PERSONAL','SISTEMA').	Habilita la Tesorería Central (RN005) y elimina la restricción que impedía cuentas múltiples (1:N).
items	Categoría como texto libre, fechas redundantes y sin control de deudas.	PK simple, FK a jugador, precio tasado, fecha adquisición, flag tiene_deuda y estado de custodia.	Cumple RN002 (un dueño por bien) y RN004 (bloqueo por gravamen o embargo).
negociaciones_tradeos	Inexistente; se pretendía ejecutar el intercambio directamente sobre transacciones.	Mesa bilateral simétrica con id_item_j1/j2, monto_j1/j2, doble confirmación y expiración.	Garantiza la máquina de estados de negociación (RN010, RN012) con soporte de permuta cruzada.
transacciones	Sobrecargada con listas de texto concatenadas (usuarios y bienes en un campo).	Cabecera inmutable de libro mayor con tipo de operación, monto nominal, fecha_hora y estado.	Estructura de auditoría contable de sólo adición (*append-only*) inalterable (RN003).
detalles_transacciones	Inexistente en el diseño preliminar.	Asientos contables por partida doble con cuenta_origen, cuenta_destino, tipo de movimiento y monto.	Trazabilidad contable estricta al centavo y cumplimiento de 1FN.

4. Estructura del Esquema (DDL)

A continuación se documenta de forma exhaustiva y rigurosa la estructura tabular de las entidades físicas en el motor MySQL (InnoDB), especificando tipos de datos, nulidad, claves, valores por defecto y su justificación técnica vinculada a las reglas de negocio:

Tabla: servidores

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_servidor	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único de la instancia del servidor.
nombre	VARCHAR(100)	NO	N/A	N/A	Nombre del servidor de RP.
porcentaje_comision	DECIMAL(5,2)	NO	CHECK (>= 0)	5.00	Porcentaje de retención/drenaje por transacción (RN006).
limite_bienes_por_jugador	INT	NO	CHECK (> 0)	100	Máximo de propiedades/vehículos permitidos por usuario (RN007).
tiempo_enfriamiento_min	INT	NO	CHECK (>= 0)	15	Minutos de espera obligatorios entre turnos laborales (RN008).
limite_tradeos_diarios	INT	NO	CHECK (> 0)	5	Cupo máximo de tradeos diarios por usuario (RN011).

Tabla: jugadores

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_jugador	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único del jugador.
id_servidor	INT	NO	FK → servidores(id_servidor)	1	Referencia al servidor al que pertenece el jugador.
nombre_usuario	VARCHAR(50)	NO	UNIQUE	N/A	Nombre de usuario o apodo único por servidor.
correo	VARCHAR(150)	NO	UNIQUE	N/A	Dirección de correo electrónico corporativo/personal.
contrasena_hash	VARCHAR(255)	NO	N/A	N/A	Hash encriptado de clave de acceso.
fecha_registro	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Marca de tiempo por defecto.
estado	VARCHAR(20)	NO	CHECK ('ACTIVO','BANCAROTA','SUSPENDIDO')	'ACTIVO'	Estados permitidos: 'ACTIVO', 'BANCAROTA', 'SUSPENDIDO' (RN009).
es_admin	BOOLEAN	NO	N/A	FALSE	Bandera de privilegios de administración.

Tabla: empleos

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_empleo	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador del empleo o rol laboral.
id_servidor	INT	NO	FK → servidores(id_servidor)	1	Referencia al servidor que habilita la plaza laboral.
nombre_empleo	VARCHAR(100)	NO	N/A	N/A	Nombre de la labor (ej. Minero, Mecánico).
tarifa_base	DECIMAL(10,2)	NO	CHECK (> 0)	N/A	Pago parametrizado por hora o unidad de trabajo (RN001).

Tabla: jornadas_laborales

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_jornada	INT	NO	PK, AUTO_INCREMENT	N/A	Código correlativo de la jornada ejecutada.
id_jugador	INT	NO	FK → jugadores(id_jugador)	N/A	Referencia al jugador que desempeñó el turno.
id_empleo	INT	NO	FK → empleos(id_empleo)	N/A	Referencia al empleo desempeñado.
horas_trabajadas	DECIMAL(6,2)	NO	CHECK (> 0)	N/A	Unidades de tiempo laboradas.
fecha_hora	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Cierre de turno registrado por defecto (RN008).
monto_pagado	DECIMAL(10,2)	NO	CHECK (>= 0)	0.00	Valor pagado inmutable (RN001, 3FN).

Tabla: cuentas

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_cuenta	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador de la cuenta.
id_jugador	INT	SÍ	FK → jugadores(id_jugador)	NULL	Vinculado si es cuenta 'PERSONAL'. Nulo en 'SISTEMA' (RN005).
id_servidor	INT	SÍ	FK → servidores(id_servidor)	NULL	Vinculado al servidor. Asignado en cuentas del Sistema.
tipo_cuenta	VARCHAR(20)	NO	CHECK ('PERSONAL','SISTEMA')	'PERSONAL'	Categorías: 'PERSONAL' o 'SISTEMA' (RN005).
saldo_inicial	DECIMAL(12,2)	NO	N/A	0.00	Fondos asignados en la apertura de la cuenta.
saldo_disponible	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Balance líquido activo para transacciones (RN009).

Tabla: items

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_item	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador único del objeto o vehículo.
id_jugador	INT	NO	FK → jugadores(id_jugador)	N/A	Relación foránea hacia el propietario legítimo (RN002).
nombre	VARCHAR(100)	NO	N/A	N/A	Nombre comercial o modelo del artículo.
precio	DECIMAL(12,2)	NO	CHECK (>= 0)	N/A	Valor de tasación original.
fecha	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Fecha de creación o adquisición.
tiene_deuda	BOOLEAN	NO	N/A	FALSE	Indicador de embargo o deuda pendiente (RN004).
antiguedad_dias	INT	NO	CHECK (>= 0)	0	Días acumulados para cálculo dinámico de depreciación.
estado_custodia	VARCHAR(20)	NO	CHECK ('PERSONAL','EMBARGADO','EN_TRADE')	'PERSONAL'	Estado de posesión física y custodia transaccional.

Tabla: negociaciones_tradeos

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_negociacion	INT	NO	PK, AUTO_INCREMENT	N/A	Código de la sesión de oferta P2P.
id_jugador_1	INT	NO	FK → jugadores(id_jugador)	N/A	Relación foránea hacia proponente inicial.
id_jugador_2	INT	NO	FK → jugadores(id_jugador)	N/A	Relación foránea hacia receptor de la oferta.
id_item_j1	INT	SÍ	FK → items(id_item)	NULL	Ítem ofrecido por el Jugador 1.
id_item_j2	INT	SÍ	FK → items(id_item)	NULL	Ítem ofrecido por el Jugador 2.
monto_j1	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Saldo líquido ofrecido por J1.
monto_j2	DECIMAL(12,2)	NO	CHECK (>= 0)	0.00	Saldo líquido ofrecido por J2.
confirmacion_j1	BOOLEAN	NO	N/A	FALSE	Confirmación del Jugador 1 (RN010).
confirmacion_j2	BOOLEAN	NO	N/A	FALSE	Confirmación del Jugador 2 (RN010).
estado	VARCHAR(50)	NO	CHECK ('PENDIENTE','ACEPTADO','EN_PROCESO','ESPERANDO_CONFIRMACION_FINAL','COMPLETADO','CANCELADO','EXPIRADO')	'PENDIENTE'	Estado de la negociación (RN012).
fecha_creacion	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Apertura de la sesión de tradeo.
fecha_expiracion	DATETIME	NO	N/A	N/A	Fecha límite calculada para completar el intercambio.

Tabla: transacciones

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_transaccion	INT	NO	PK, AUTO_INCREMENT	N/A	Identificador inmutable de transacción.
id_item_afectado	INT	SÍ	FK → items(id_item)	NULL	Relación foránea hacia el ítem transferido (si aplica).
id_negociacion	INT	SÍ	FK → negociaciones_tradeos(id_negociacion)	NULL	Relación foránea hacia la mesa de tradeo origen.
id_jornada	INT	SÍ	FK → jornadas_laborales(id_jornada)	NULL	Relación foránea hacia la jornada laboral origen.
tipo_transaccion	VARCHAR(30)	NO	CHECK ('PAGO_SALARIO','TRADEO_P2P','COMPRA_COMERCIANTE','AJUSTE_ADMIN')	N/A	Categorías: 'PAGO_SALARIO', 'TRADEO_P2P', 'COMPRA_COMERCIANTE'.
estado_transaccion	VARCHAR(20)	NO	CHECK ('COMPLETADA','CANCELADA','REVERTIDA')	'COMPLETADA'	Estados: 'COMPLETADA', 'CANCELADA', 'REVERTIDA' (RN003).
monto	DECIMAL(12,2)	NO	CHECK (>= 0)	N/A	Valor nominal total procesado.
fecha_hora	DATETIME	NO	N/A	CURRENT_TIMESTAMP	Marca de tiempo inmutable de ejecución (RN003).

Tabla: detalles_transacciones

COLUMNA	TIPO DE DATO	NULO	CLAVE / CONSTRAINT	DEFAULT	OBSERVACIONES Y REGLA VINCULADA
id_detalle	INT	NO	PK, AUTO_INCREMENT	N/A	Código único del asiento contable.
id_transaccion	INT	NO	FK → transacciones(id_transaccion)	N/A	Relación foránea hacia la transacción cabecera.
cuenta_origen	INT	NO	FK → cuentas(id_cuenta)	N/A	Relación foránea hacia la cuenta debitada.
cuenta_destino	INT	NO	FK → cuentas(id_cuenta)	N/A	Relación foránea hacia la cuenta acreditada.
tipo_movimiento	VARCHAR(10)	NO	CHECK ('DEBITO','CREDITO')	N/A	Categorías de partida doble: 'DEBITO' o 'CREDITO'.
monto_detalle	DECIMAL(12,2)	NO	CHECK (> 0)	N/A	Importe del movimiento contable parcial.
concepto	VARCHAR(80)	SÍ	N/A	NULL	Descripción corta o justificación contable.

5. Informe de Mejoras y Normalización

5.1 Mejoras Implementadas

- **Garantía de Cuenta Única de Sistema por Servidor:** La adición de la cuenta con tipo_cuenta = 'SISTEMA' (id_jugador NULL) impidió la duplicación de tesorerías por servidor sin interferir con las cuentas personales de los jugadores.
- **Atomicidad mediante Procedimientos y Bloques Transaccionales:** Los flujos complejos de pago salarial y confirmación de tradeo fueron empaquetados en transacciones explícitas (START TRANSACTION, COMMIT, ROLLBACK). En caso de cualquier error (falta de fondos, deudas o expiración), los manejadores restauran la base de datos a su estado previo.
- **Inclusión de Disparadores y Restricciones CHECK:** Validación de límites de inventario, tiempos de enfriamiento laboral e inmutabilidad estricta del libro mayor contable.
- **Control de Expiración Automática:** Gestión del ciclo de vida para marcar ofertas como 'EXPIRADO' cuando sobrepasan su tiempo límite.

5.2 Normalizaciones Realizadas (1FN a 4FN / DKNF)

FORMA NORMAL	CRITERIO TEÓRICO APLICADO	RESOLUCIÓN Y GARANTÍA EN ECONOMY RP
1FN (Atomicidad)	Cada celda contiene un único valor escalar indivisible; sin atributos multivaluados ni repetidos.	Se descompuso la lista de participantes en detalles_transacciones (una cuenta origen y una destino por fila) y los bienes en items y negociaciones_tradeos.
2FN (Dependencia Total)	Cumple 1FN y ningún atributo no-clave depende parcialmente de claves compuestas.	Todas las tablas del sistema utilizan claves primarias simples subrogadas (id_jornada, id_cuenta, etc.), anulando dependencias parciales.
3FN (Sin Transitividad)	Cumple 2FN y ningún atributo no-clave depende transitivamente de otro atributo no-clave.	En jornadas_laborales se sustituyó el cálculo dinámico de ganancia por monto_pagado inmutable, evitando depender de cambios futuros en la tarifa base de empleos.
FNBC / BCNF	Todo determinante funcional en el esquema constituye formalmente una superclave.	Se eliminaron dependencias encubiertas soportando todas las relaciones estrictamente sobre claves primarias enteras o índices UNIQUE declarados.
4FN (Multivaluadas)	No existen dependencias multivaluadas no triviales independientes entre sí.	El historial laboral, las cuentas monetarias y los ítems de un jugador residen en tablas independientes vinculadas unívocamente por la FK id_jugador.
DKNF (Dominio/Clave)	Toda restricción es consecuencia directa de la definición de dominios y restricciones de clave.	Todas las reglas de negocio se imponen nativamente mediante restricciones CHECK, NOT NULL, UNIQUE y FOREIGN KEY.

6. Pruebas y Hallazgos

Para asegurar la robustez de la base de datos en MySQL (InnoDB), se ejecutó una suite exhaustiva de pruebas de inserción y manejo de errores:

6.1 Errores Encontrados

ERROR 1: INSERT INTO servidores ... servidores... Error Code: 1064

Falla Reportada (Error): Error Code: 1064. You have an error in your SQL syntax; check the manual near 'servidores' at line 1.

Análisis del Problema / Causa: Se incluyó accidentalmente la palabra residual 'servidores' al final de la instrucción SQL.

Cómo se corrigió: Se eliminó la palabra residual dejando únicamente las instrucciones SQL válidas.

Código corregido / Verificación:

```
INSERT INTO servidores (nombre, porcentaje_comision, limite_bienes_por_jugador, tiempo_enfriamiento_min) VALUES ('Servidor Prueba', 5.00, 10, 1);
```

ERROR 2: INSERT INTO cuentas (id_servidor, tipo_cuenta...) VALUE... Error Code: 1062

Falla Reportada (Error): Error Code: 1062. Duplicate entry '1' for key 'cuentas.uq_cuenta_sistema_por_servidor'

Análisis del Problema / Causa: El gestor bloqueó la inserción porque la cuenta de sistema para ese servidor ya existía.

Cómo se corrigió: Se procedió a verificar la existencia de la cuenta ejecutando la consulta de lectura.

Código corregido / Verificación:

```
SELECT * FROM cuentas WHERE id_servidor = 1 AND tipo_cuenta = 'SISTEMA';
```

ERROR 3: INSERT INTO jugadores (id_servidor, nombre_usuario, niv... Error Code: 1054

Falla Reportada (Error): Error Code: 1054. Unknown column 'nivel' in 'field list'

Análisis del Problema / Causa: El gestor indica que se intentó guardar en la columna 'nivel', pero dicha columna no existe en la estructura física de la tabla jugadores.

Cómo se corrigió: Se verificó la estructura de la tabla y se eliminó la columna nivel de la consulta de inserción.

Código corregido / Verificación:

```
INSERT INTO jugadores (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (1, 'Usuario_Prueba', 'user@rp.com', 'hash123');
```

ERROR 4: INSERT INTO jugadores (id_servidor, nombre_usuario...) ... Error Code: 1452

Falla Reportada (Error): Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (fk_jugador_servidor).

Análisis del Problema / Causa: La regla de Foreign Key exige que un jugador no puede existir sin estar asignado a un servidor real registrado.

Cómo se corrigió: Se captura dinámicamente el ID real del servidor creado previamente.

Código corregido / Verificación:

```
INSERT INTO jugadores (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (@id_servidor_real, 'Jugador_Alpha', 'alpha@email.com', 'hash_secret_123');
```

ERROR 5: INSERT INTO cuentas (id_jugador, id_servidor...) VALUES... Error Code: 1452

Falla Reportada (Error): Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (fk_cuenta_jugador).

Análisis del Problema / Causa: La base de datos buscó al jugador con id_jugador = 9999 y no lo encontró.

Cómo se corrigió: Se utiliza la variable dinámica con el ID real asignado al jugador.

Código corregido / Verificación:

```
INSERT INTO cuentas (id_jugador, id_servidor, tipo_cuenta, saldo_inicial, saldo_disponible) VALUES (@id_jugador_real, 1, 'PERSONAL', 5000.00, 5000.00);
```

ERROR 6: INSERT INTO empleos (id_servidor, nombre_empleo...) VAL... Error Code: 1452

Falla Reportada (Error): Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (fk_empleo_servidor).

Análisis del Problema / Causa: Al intentar asignarle el empleo a un servidor inexistente, MySQL no lo encuentra y bloquea la acción.

Cómo se corrigió: Se ejecuta la consulta usando la variable dinámica para atrapar el ID real del servidor.

Código corregido / Verificación:

```
INSERT INTO empleos (id_servidor, nombre_empleo, tarifa_base) VALUES (@id_servidor_real, 'Minero', 150.00);
```

6.2 Pruebas de Validación de Integridad y Lógica Transaccional

PRUEBA 8: Horas trabajadas negativas

Objetivo de la Prueba: Comprobar que sp_pagar_jornada rechaza horas inválidas (cero o negativas).

Código de la prueba:

```
CALL sp_pagar_jornada(@ja, @emp, -10);
```

Resultado esperado: Error de validación; ningún saldo cambia.

Diagnóstico / Riesgo: Si no se controla, un valor negativo generaría montos pagados negativos, drenando saldo indebido al jugador.

Código corregido / Blindaje:

```
IF NEW.horas_trabajadas <= 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Las horas trabajadas deben ser mayores a cero'; END IF;
```

PRUEBA 9: Robo de ítems en un tradeo

Objetivo de la Prueba: Verificar que un jugador solo pueda ofrecer ítems que le pertenecen legítimamente.

Código de la prueba:

CALL sp_abrir_negociacion(@ja, @jb, @item_c, NULL, 0, 0, 10); -- @item_c pertenece a C

Resultado esperado: Error al abrir la negociación, y el bien sigue perteneciendo a su dueño original.

Diagnóstico / Riesgo: Garantiza que ningún bien pueda ser negociado por un tercero no autorizado.

Código corregido / Blindaje:

```
IF NOT EXISTS (SELECT 1 FROM items WHERE id_item = p_id_item_j1 AND id_jugador = p_id_jugador_1) THEN SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'El ítem ofrecido no le pertenece al jugador 1'; END IF;
```

PRUEBA 10: Confirmar una negociación ya expirada

Objetivo de la Prueba: Comprobar que no se puede confirmar un tradeo fuera de tiempo.

Código de la prueba:

CALL sp_confirmar_tradeo(@id_negociacion_expirada, @ja);

Resultado esperado: Error 'negociación expirada' y transición del estado a 'EXPIRADO'.

Diagnóstico / Riesgo: Evita que se ejecuten tratos obsoletos cuyos términos ya no son válidos para ambas partes.

Código corregido / Blindaje:

```
IF v_fecha_exp < NOW() THEN UPDATE negociaciones_tradeos SET estado = 'EXPIRADO' WHERE id_negociacion = p_id; SIGNAL SQLSTATE
'45000' SET MESSAGE_TEXT = 'La negociacion ya expiro'; END IF;
```

PRUEBA 11: Un tradeo salta el límite de bienes

Objetivo de la Prueba: Verificar que el límite del servidor (ej. 2 bienes) también se respeta cuando se reciben ítems por tradeo.

Código de la prueba:

CALL sp_confirmar_tradeo(@neg, @jd); -- D ya tiene 2 bienes

Resultado esperado: El control falla con 'límite máximo de bienes'; el tradeo se cancela y D sigue con 2 ítems.

Diagnóstico / Riesgo: Evita el acaparamiento de activos en una sola cuenta mediante transferencias entre jugadores.

Código corregido / Blindaje:

```
IF v_cant_j1 - (v_item_j1 IS NOT NULL) + (v_item_j2 IS NOT NULL) > v_limite THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'El
tradeo excede el limite de bienes'; END IF;
```

PRUEBA 12: Tradeo de un jugador consigo mismo

Objetivo de la Prueba: Comprobar que no se puede abrir una negociación donde ambos participantes son el mismo jugador.

Código de la prueba:

CALL sp_abrir_negociacion(@ja, @ja, NULL, NULL, 100, 0, 10);

Resultado esperado: Error al abrir la negociación y bloqueo de la inserción.

Diagnóstico / Riesgo: Evita el auto-bloqueo de sesiones y el colapso de la máquina de estados de doble confirmación.

Código corregido / Blindaje:

```
CONSTRAINT chk_negociacion_distintos CHECK (id_jugador_1 <> id_jugador_2)
```

7. Defensa Técnica y Sustentación del Esquema

La arquitectura del esquema DDL de Economy RP en MySQL 8.0 (InnoDB) no es un repositorio pasivo, sino un motor transaccional gobernado por reglas estrictas de integridad relacional. A nivel de defensa técnica, se sustenta el comportamiento del sistema respondiendo a las preguntas fundamentales de flujo de información:

1. ¿Por qué el modelo restringe el traspaso de bienes en ciertas condiciones?

El modelo bloquea de forma intransigente la transferencia de cualquier ítem que registre gravámenes (tiene_deuda = TRUE, RN004) o cuyo titular no coincida con el operador autenticado (items.id_jugador != emisor, RN002). Asimismo, el motor rechaza la transacción si el receptor ya alcanzó su cuota máxima de inventario (RN007) o si no dispone de liquidez suficiente para cubrir el importe acordado más el 5% de comisión tributaria (RN006). Estas restricciones protegen a la comunidad de estafas, venta fraudulenta de vehículos embargados y sobregiros contables.

2. ¿Por qué el modelo permite la emisión de dinero sólo a través de canales específicos?

El esquema prohíbe la creación de dinero 'de la nada' mediante inserciones directas en cuentas personales. Todo flujo de entrada a la economía debe ser respaldado por una jornada laboral completada (jornadas_laborales) y registrado como un débito formal a la cuenta oficial de Tesorería del SISTEMA (RN005). Esto asegura que la masa monetaria en circulación sea 100% auditable mediante la suma simétrica de débitos y créditos en detalles_transacciones.

3. ¿Por qué se implementó Inmutabilidad Contable por Partida Doble?

En un entorno financiero virtual, la modificación arbitraria de saldos mediante sentencias UPDATE destruye la trazabilidad histórica. Al estructurar transacciones y detalles_transacciones bajo un patrón de sólo adición (*append-only*), cualquier ajuste administrativo debe realizarse mediante un nuevo asiento compensatorio, protegiendo la fe pública y la confianza de los participantes.

4. Garantía de Propiedades ACID en el Motor Relacional:

- **Atomicidad (A):** Todas las operaciones de intercambio P2P y liquidación laboral se ejecutan en transacciones explícitas con manejadores EXIT HANDLER FOR SQLEXCEPTION ROLLBACK, garantizando que todo se aplique en su totalidad o se revierta por completo.
- **Consistencia (C):** El cumplimiento riguroso de 1FN a 4FN, las llaves foráneas y las restricciones CHECK aseguran que la base de datos nunca quede en un estado inválido.
- **Aislamiento (I):** Las consultas de verificación emplean bloqueos pesimistas (SELECT ... FOR UPDATE), evitando condiciones de carrera en operaciones concurrentes.
- **Durabilidad (D):** Los registros persistidos en MySQL con el motor transaccional InnoDB garantizan la recuperación total ante fallos de hardware o caídas de red.

8. Anexo: Modificaciones y Resoluciones tras Retroalimentación Sincrónica

A continuación se sintetizan formalmente las mejoras aplicadas a partir de las observaciones del docente asesor en la sesión sincrónica:

OBSERVACIÓN DOCENTE	RIESGO RELACIONAL DIAGNOSTICADO	SOLUCIÓN DDL IMPLEMENTADA
Punto 1: Estructura de Ítems y Posesión Rígida	FK id_jugador rígida (1:N) impedía que los bienes pertenecieran a concesionarios o quedaran bloqueados en tradeo sin perder al dueño legítimo.	Se modularizó con id_jugador INT NULL y se incorporaron estado_custodia CHECK ('PERSONAL','EMBARGADO','EN_TRADE') y tipo_propietario CHECK ('JUGADOR','CONCESIONARIO','SISTEMA').
Punto 2: Cardinalidad de Cuentas Financieras	Restricción UNIQUE en id_jugador forzaba relación 1:1, impidiendo cuentas múltiples (ahorro, comercial, corporativa).	Se eliminó la restricción UNIQUE en cuentas.id_jugador, habilitando formalmente cardinalidad 1:N clasificada mediante tipo_cuenta CHECK (...).
Punto 3: Atomicidad y Simetría Contable	Riesgo de descuadre contable si la simetría débito/crédito dependía únicamente de inserciones externas.	Se blindó mediante bloques transaccionales atómicos ACID con bloqueos pesimistas en InnoDB, asegurando SUM(Débitos) = SUM(Créditos) con ROLLBACK automático ante cualquier falla.
Punto 4: Estandarización de Nomenclatura	Uso de prefijo T_ y singularidades inconsistentes.	Se normalizó el DDL completo eliminando el prefijo T_ y estandarizando nombres de tablas en plural (servidores, jugadores, etc.) en snake_case minúscula.

Miguel Ángel Cardona Agudelo
Desarrollo DB & DDL

Luisa María López Orrego
Normalización & Calidad

Jorge Luis Ordoñez Ávila
Pruebas & Auditoría